

Foundry测试

在 Hardhat 项目里整合 Foundry

Hardhat 的脚本环境（特别是对于复杂的部署），但也认识到使用 Foundry 进行测试和模糊处理的好处。在同一个代码库中使用这两个应用程序，可以提供两个最佳选择

1. 按照前面的基本命令，先创建一个 Hardhat 项目
2. 执行命令 `npm i --save-dev @nomicfoundation/hardhat-foundry` 安装插件
3. 在 `hardhat.config.js` 文件顶部引入插件，如下所示：

```
require("@nomicfoundation/hardhat-toolbox");  
require("@nomicfoundation/hardhat-foundry");
```

1. 执行命令 `npx hardhat init-foundry` 会生成一个 `foundry.toml` 文件和安装 `forge-std` 标准库
2. `foundry.toml` 将 `hardhat` 项目中的 `node_modules` 和 `contracts` 目录进行导入 `foundry`

foundry目录结构

项目下会新增以下目录与文件：

- ├─ foundry.toml
- ├─ lib 依赖的包文件目录
- ├─ script 自定义的脚本目录
- ├─ src 合约源码目录
- └─ test 测试脚本目录

Foundry 使用 Git submodule 来管理依赖库，`.gitmodules` 文件记录了目录与子库的关系

工具链

- **forge**: 用来执行初始化项目、管理依赖、测试、构建、部署智能合约；
- **cast**: 执行以太坊 RPC 调用的命令行工具, 进行智能合约调用、发送交易或检索任何类型的链数据
- **anvil**: 创建一个本地测试网节点, 也可以用来分叉其他与 EVM 兼容的网络。

优势

- 全部使用 solidity 语言进行开发和编写测试和自定义脚本
- 作弊码功能强大
- 与链上交互能力强
- cast 工具
- anvil 分叉链
- 内置的模糊测试
- 硬件钱包兼容

- 利用快照和Gas报告更好的进行Gas优化
- 静态分析

劣势

- 相对js, 定制能力较差, 比如我们合约部署工具, js脚本根据入参参数化进行合约部署、升级、验证

测试

在测试目录下 `test` 添加自己的测试用例, foundry 测试用例使用 `.t.sol` 后缀, 约定具有以 `test` 开头的函数的合约都被认为是一个测试

编写测试要导入并继承Forge 标准库(forge-std) 的 Test 合约

该合约本身是DappTools框架的[DSTest](#) (提供基本的日志记录和断言功能) 的超集

`setUp` 函数是在每一个测试函数执行前都会执行的启动函数, 所有测试函数都是隔离的, 每个测试函数都使用 `setUp` 之后的状态执行, 并在其自己的独立 EVM 中执行。

Forge 标准库它提供了开始编写测试所需的所有基本功能, 包含:

- `vm.sol`: 最新的作弊码接口
- `console.sol` 和 `console2.sol`: Hardhat 风格的日志记录功能
- `Script.sol`: [Solidity 脚本](#) 的基本实用程序
- `Test.sol`: DSTest 的超集, 包含标准库、作弊码实例 (`vm`) 和 Hardhat 控制台
- `StdAssertions.sol`: 扩展了 [DSTest](#) 库中的断言函数
- `StdError.sol`: 提供围绕常见内部 Solidity 错误 errors 和回退 reverts 的包装器
- `StdStorage.sol`: 使操作合约存储变得容易。它可以找到并写入与特定变量关联的存储槽
- `StdMath.sol`: 包含 Solidity 中未提供的有用的数学函数
-

注意: `import "forge-std/Test.sol";` 导入所有

`import {Test} from "forge-std/Test.sol";` 只导入Test合约, 不包含console和vm等等

注意: `console2.sol` 包含 `console.sol` 的补丁, 允许 Forge 解码控制台调用的跟踪, 但它与 `Hardhat` 不兼容。

执行测试命令

```
forge test
```

日志详细程度

- `-vv` 显示 `console.log` 输出。
- `-vvv` 显示失败测试的执行跟踪。
- `-vvvv` 显示所有测试的执行跟踪, 并显示失败测试的设置 (setup) 跟踪。
- `-vvvvv` 显示所有测试的执行和设置 (setup) 跟踪。

运行特定测试：

- `--match-test` 运行与指定正则表达式匹配的测试函数。
- `--match-contract` 运行与指定正则表达式匹配的测试合约。
- `--match-path` 运行与指定路径匹配的源文件中的测试。

测试覆盖率

`forge coverage`

`--report` 允许您指定用于覆盖率的报告类型。此标志可以多次使用。

它有三个不同的选项，默认设置为 `summary`。

`summary`

输出一个图表，显示您的代码有多少百分比被测试覆盖。

`lcov`

在项目目录的根目录中创建一个包含覆盖率数据的 `lcov.info` 文件。

`debug`

输出描述未覆盖代码位置的行。

分叉测试

Forge 支持使用两种不同的方法在分叉环境中进行测试：

- [分叉模式 \(Forking Mode\)](#) — 通过 `forge test --fork-url` 标志使用一个单独分叉进行所有测试，

`--fork-block-number` 指定要从中分叉的区块高度，

`--etherscan-api-key <your_etherscan_api_key>` 使用 Etherscan 在分叉环境中识别合约源码
当需要与现有合约进行交互时，分叉特别有用

- [分叉作弊码 \(Forking Cheatcodes\)](#) — 通过 [forking作弊码](#) 在 Solidity 测试代码中直接创建、选择和管理多个分叉

在测试中使用多个分叉，每个分叉都通过其自己唯一的 `uint256` 标识符进行识别，分叉是相互独立的

1. 在 `setUp` 期间使用 [createFork](#) 创建的多个分支，每个 fork 都有一个唯一标识符 (`uint256 forkId`)，该标识符在首次创建时分配
2. 在测试函数中通过 [selectFork](#) 指定 `forkId` 来启用特定的分叉（当前活动分叉的标识符可以通过 [activeFork](#) 检索）

分叉数据缓存在 `~/.foundry/cache/rpc/<chain name>/<block number>` 中。要清除缓存，只需删除目录或运行 [forge clean](#)（删除所有构建工件和缓存目录）。

只有 `msg.sender` 和启动后部署的本地测试合约（`ForkTest`）的账户数据是持久的，在多个分叉中可以共享，其他远程数据和在选择分叉后生成的数据都不是共享的

可以使用 `makePersistent` 作弊码将任意账户变成持久帐户，*persistent* 帐户是唯一的，即它存在于所有分叉上，就可在多个分叉中共享数据了

模糊测试

通过向程序输入大量随机数据或畸形数据来发现潜在的漏洞。对于智能合约来说，模糊测试可以帮助我们发现合约中可能存在的越界访问、算术溢出、重入攻击等安全问题。

在测试函数的形参定义要使用的模糊输入数据类型

使用 `assume` 作弊码排除某些情况。模糊器 fuzzer 将丢弃输入并开始运行新的模糊测试：

例 `vm.assume(amount > 0.1 ether)`

- "runs" 是指模糊器 fuzzer 测试的场景数量。默认情况下，模糊器 fuzzer 将生成 256 个场景，但用户可以设置此参数以及其他测试执行参数。有关模糊测试器配置详细信息，请参阅 [这里](#)。
- "μ"（希腊字母 mu）是所有模糊运行中使用的平均 Gas
- "~"（波浪号）是所有模糊运行中使用的中值 Gas

不变性测试

检查在多次合约交互（如调用多个函数、交易等）后，某些状态变量或合约条件仍然符合预期（例如合约余额不降低、token总供应量、某个变量不会被意外更改等）。这种测试方法可以有效地捕获意外的状态变化或漏洞

在进行每个函数调用后，都会对所有定义的不变性进行断言

通过在函数名前加上 `invariant` 前缀来表示不变性测试（例如，`function invariant_A()`）

Gas报告

编译时的合约Gas报告可以通过foundry.toml配置来设置

```
为特定合约生成报告： gas_reports = ["MyContract", "MyContractFactory"]
为所有合约生成报告： gas_reports = ["*"]
```

`forge test --gas-report` 执行命令。

Gas快照

优化函数的一个方法是使用测试合约生成 Gas 快照，并在修改前后进行快照对比：

```
forge snapshot --snap gas1.txt
```

这将提供之前的Gas报告和当前快照之间的差异。

- `--asc` 选项将结果按升序 排序
- `--desc` 将结果按降序排序。

`--match-path` 可以使用 `forge test` 中的匹配参数

比较Gas用量

```
forge snapshot --diff gas1.txt
```

foundry.toml 配置

可以定义多个配置命名空间

`[profile.default]` 是基础配置，所有其他配置文件都继承自该配置

```
solc_version = "0.8.20"
remappings = [
    "@solmate-utils/=lib/solmate/src/utils/",
]
# 重映射
fuzz_runs = 100
# Fuzz 测试运行的次数，默认 100，可以调整以提高测试覆盖率
rpc_url = "${RPC_URL}"
# 使用环境变量设置 RPC URL
private_key = "${PRIVATE_KEY}"
# 使用环境变量设置私钥
verbosity = 3
# 设置日志详细程度，范围从 0 到 5，数字越大输出越详细
tx_gas_limit = 15000000
# 设置单次交易的 gas 限制
tx_gas_price = 20000000000
# 设置交易的 gas 价格（单位：wei）
deployer = "0xYourDeployerAddressHere"
# 指定部署者地址
```

作弊码

重要的作弊代码有：

[Environment](#): 改变以太坊虚拟机状态的作弊码

- `vm.warp(uint256)`: 设置 `block.timestamp`
- `vm.roll(uint256)`: 设置 `block.height`
- `vm.chainId(uint256)`: 设置 `block.chainid`
- `vm.store(address account, bytes32 slot, bytes32 value)`: 在账户 `account` 的存储槽 `slot` 中存储值 `value`
- `vm.load(address account, bytes32 slot)`: 从账户 `account` 的存储槽 `slot` 中加载值
- `vm.deal(address, uint256)`: 设置一个地址的余额
- `vm.prank(address)`: 将 `msg.sender` 设置为 指定地址 用于下一次调用
- `vm.startPrank(address)`: 设置 `msg.sender` 用于所有后续调用，直到调用 `stopPrank` 为止
- `vm.startPrank(address sender, address origin)`: 为所有后续调用设置
- `vm.stopPrank()`: 停止由 `startPrank` 启动的活动 `Prank`，将 `msg.sender` 和 `tx.origin` 重置为调用 `startPrank` 之前的值
- `vm.readCallers()` external returns (CallerMode callerMode, address msgSender, address txOrigin): 读取当前的 `CallerMode`，`msg.sender` 和 `tx.origin`

- `vm.record()`：告诉虚拟机开始记录所有存储读取和写入操作。要访问读取和写入操作，请使用 `accesses`
- `vm.accesses(address)`：获取在地址上已读取（`reads`）或已写入（`writes`）的所有存储槽
- `vm.recordLogs()`：告诉虚拟机开始记录所有已发出的事件。要访问它们，请使用 `getRecordedLogs`
- `vm.getRecordedLogs()` external returns (`Log[] memory`)：获取由 `recordLogs` 记录的发出的事件。
- 调用此函数将消耗记录的日志
- `vm.setNonce(address account, uint64 nonce)`：设置给定账户的 `nonce`，新的 `nonce` 必须高于账户当前的 `nonce`
- `vm.getNonce(address account)` external returns (uint64)：获取给定账户或 `钱包` 的 `nonce`
- `vm.getNonce(Wallet memory wallet)` external returns (uint64)：获取给定账户或 `钱包` 的 `nonce`
- `vm.mockCall(address where, bytes calldata data, bytes calldata retdata)`：如果调用数据严格或宽松匹配 `data`，则模拟对地址 `where` 的所有调用，并返回 `retdata`
- `vm.mockCall(address where, uint256 value, bytes calldata data, bytes calldata retdata)`：我们可以模拟具有特定 `msg.value` 的调用。在存在歧义的情况下，`calldata` 匹配优先于 `msg.value`

[Assertions](#): 断言作弊码

- `vm.expectRevert()`：断言下一次调用会回滚，无论消息是什么
- `vm.expectRevert(bytes4 message)`：断言下一次调用会以指定的 4 个字节回滚
- `vm.expectRevert(bytes calldata message)`：断言下一次调用会以指定的字节回滚，可以选择字符串、选择器、ABI 编码（`abi.encodeWithSelector()`）
- `vm.expectEmit(true, false, false, false, address emitter); emit Transfer(address(this)); transfer();` 在下次调用期间断言特定日志被发出
 - 调用作弊码，指定我们是否应检查第一个、第二个或第三个主题，以及日志数据（`expectEmit()` 会检查它们全部）。主题 0 总是被检查。
 - 发出我们在下次调用中应该看到的事件。
 - 执行调用。
 - 可以多次执行步骤 1 和 2，以匹配下次调用中的事件序列，在发出大量事件的函数中，有可能“跳过”事件并且只匹配特定序列，但是这个检查序列必须始终保持顺序
 - `emitter` 参数可选，检查发出地址

[Fuzzer](#): 配置模糊器的作弊码

- `vm.assume(bool)`：如果布尔表达式计算结果为 `false`，则模糊器将丢弃当前的模糊输入并开始新的模糊运行

[External](#): 与外部状态（文件、命令等）交互的作弊码

- `vm.sleep(uint256 milliseconds)`：休眠指定的毫秒数
- `vm.envOr(key, defaultValue)`：用于读取任何类型的环境变量：如果请求的环境键不存在，`envOr()` 将返回默认值
- `vm.envBool`、`envUint`。。。：读取不同类型的环境变量

[Utilities](#): 较小的实用程序作弊码

- `vm.addr(uint256 privateKey)` external returns (address)：计算给定私钥的地址

- `vm.sign(uint256 privateKey, bytes32 digest)` external returns (uint8 v, bytes32 r, bytes32 s): 使用私钥 `privateKey` 或 `Wallet` `wallet` 对摘要 `digest` 进行签名, 返回 `(v, r, s)`
- `vm.sign(Wallet memory wallet, bytes32 digest)` external returns (uint8 v, bytes32 r, bytes32 s): 这对于测试需要签名数据并执行 `ecrecover` 以验证签署者的函数非常有用
- `vm.skip(bool skip)`: 条件性地将测试标记为已跳过。必须在测试的顶部调用它, 以确保在没有任何执行的情况下跳过测试。
- `vm.label(address addr, string calldata label)`: 在测试跟踪中为 `addr` 设置标签 `label`, 如果一个地址被标记, 标签将显示在测试跟踪中, 而不是地址
- `vm.deriveKey(string calldata mnemonic, uint32 index )` external returns (uint256): 从给定的助记词或助记词文件路径派生私钥
- `vm.deriveKey(string calldata mnemonic, string calldata path, uint32 index )` external returns (uint256): `path`指定派生路径, 如"`m/44'/60'/0'/1/`"
- `vm.parseBytes`、`parseAddress`、`parseUint`.....: 将 `string` 的值解析为各种类型
- `vm.toString()`: 将任何类型转换为其字符串版本
- `breakpoint(string)`: 在调试器视图中设置断点
- `vm.createWallet(string calldata)` external returns (Wallet memory): 当给定一个参数来派生私钥时, 创建一个新的钱包结构
- `vm.:`

[Forking](#): 分叉模式作弊码

[Snapshots](#): 快照作弊码

- `vm.snapshot()` external returns(uint256): 对区块链的状态进行快照, 并返回创建的快照的标识符
- `vm.revertTo(uint256)` external returns(bool): 将区块链的状态回滚到给定的快照。这将删除给定的快照, 以及在其之后创建的任何快照

[RPC](#): 与 RPC 相关的作弊码

- `vm.rpc(string calldata method, string calldata params)`: 访问在 `foundry.toml` 的 `rpc_endpoints` 对象中配置的所有 RPC 端点

[File](#): 用于处理文件的作弊码

[Assertions](#):

- `fail(string memory err)`: 测试失败, 并发出消息
- `assertEq(bool a, bool b, string memory err)`: 断言 `a` 等于 `b`, 对 `bool`、`bytes`、`int256` 和 `uint256` 数组起作用, `err`可选
- `assertFalse(bool data)`: 断言 条件 为假
- 创建自定义断言

```
function myAssertion(自定义断言条件) {
    if (自定义断言逻辑) {
        emit log_string("");
        fail();
    }
}
```

[Cheats](#)

- `rewind(uint256 time)` : 减少指定秒数到 `block.timestamp`

[Std Errors](#): 对一些错误进行了封装

[Std Math](#): 对一些数学计算进行了封装

console

`console.log` 实现了与 Hardhat 的 `console.log` 中相同的格式化选项。

需要导入 `forge-std/console.sol`

- 例子: `console.log("Changing owner from %s to %s", currentOwner, newOwner)`
- 可以用以下类型的任何顺序的最多 4 个参数调用 `console.log`:
 - `uint`
 - `string`
 - `bool`
 - `address`

`console.log` 是用标准的 Solidity 实现的, 它兼容 Anvil 和 Hardhat 网络

`console.log` 调用可以在其他网络中运行, 如 mainnet、kovan、ropsten 等。它们在这些网络中什么都不做, 但确实花费了极少的 Gas。

Cast

我们可以 `call` 方式调用合约请求链上数据。也可以提供凭证 (私钥) 来发送一个交易

`cast receipt $TX_HASH`: 获取一个交易的交易收据。

`cast send --ledger vitalik.eth --value 0.1ether`: 用你的 Ledger 给 Vitalik 发送一些 ether

`cast send --ledger 0x... "deposit(address,uint256)" 0x... 1`: 在一个合约上调用 `deposit(address token, uint256 amount)`

`cast call 0xc02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2 "balanceOf(address)(uint256)"`

`0x...`: 在不发布交易的情况下对账户进行调用

`--trace`

打印交易的跟踪信息。

`--debug`

使用交易的交互式调试器。需要 `--trace`。

`--verbose`

打印更详细的跟踪信息。需要 `--trace`。

`--labels <address:label>`

要应用于跟踪的标签, 格式为 `address:label`。需要 `--trace`。

`cast tx $TX_HASH`: 获得有关交易的信息

`cast estimate 0xabc123 "mint(uint256)" 3`: 估算一个 Gas 成本


```
cast logs --from-block 15537393 --to-block latest 'Transfer (address indexed from, address indexed to, uint256 value)' 0x2e8ABfE042886E4938201101A63730D04F160A82: 使用
```

解码原始的 calldata

[illegible]

编码 calldata

```
cast calldata "someFunc(address,uint256)" 0x... 1
```

Anvil

```
anvil --fork-url https://mainnet.infura.io/v3/$INFURA_KEY 分叉一个网络
```

后续可以使用cast进行交互

其他命令

forge doc - 文档生成器

`--build` 从生成的文件构建 `mdbook`。

`--serve` 在本地提供文档网页。

`--port` 端口 用于提供文档网页的端口。需要 `--serve`。

```
forge flatten src/Contract.sol
```

扁平化,其中包括外部合约的依赖关系合并到一个文件中

```
--output file_name
```

输出扁平化合约的路径。如果不指定，扁平化的合约将被输出到stdout。

`forge-config` - 显示当前的配置。

`forge-tree` - 显示项目的树状可视化的依赖关系图。

`forge-build` - 构建项目的智能合约。

forge-script - 以脚本形式运行智能合约，建立可在链上发送的交易。

```
--broadcast
```

广播交易。

```
forge inspect src/MyContract.sol abi 生成ABI
```

ABI转换成solidity接口

<https://gnidan.github.io/abi-to-sol/>

最佳实践

1. 要测试 `internal` 函数，请编写一个继承自被测合约 (CuT) 的工具合约。从 CuT 继承的工具合约将 `internal` 函数暴露为 `external` 函数。
2. 要测试 `private` 函数，因为它们不能被任何其他合约访问。可选如下：
 - 将 `private` 函数转换为 `internal`。
 - 将逻辑复制/粘贴到您的测试合约中，并编写一个在 CI 检查中运行的脚本，以确保两个函数相同。
3. 编写正面和负面的单元测试
4. 私钥管理
 - 使用硬件钱包。Ledger 和 Trezor 等硬件钱包将种子短语存储在安全隔离区中
 - 环境变量存储私钥
 - 使用多签
 - 合约部署工具管理私钥。托管钱包

5. 注释

对于公共或外部方法和变量，使用 [NatSpec](#) 注释。

- `forge doc` 将解析这些以自动生成文档。
- Etherscan 将在合约 UI 中显示它们。
- 对于复杂的 NatSpec 注释，考虑使用像 [PlantUML](#) 这样的工具来生成 ASCII 艺术图，以帮助解释代码库的复杂方面。

静态分析

...